

週知快報

私人研究週刊

論文 5 篇 · 專案 5 個 · 文章 4 篇

01 本週焦點

記憶即技能

當AI學會自己管理記憶，模型規模不再是唯一解方

一個32B模型在NetHack裡一次次走回同一條死巷，不是因為打不過怪物，而是記不住三十步前已經試過的路。Stanford團隊的AutoMem[1]想解決的正是這個問題：記憶管理長期被當成固定架構模組手工設計，而不是一項可以被訓練、被優化的技能。

他們讓agent自己決定何時讀寫記憶，再用meta-LLM審閱完整執行紀錄、改寫記憶schema，並篩選表現最好的記憶決策訓練出獨立的LoRA記憶專家。結果是，僅優化記憶就讓32B模型在MiniHack上的過關率翻了3.67倍，甚至超越同系列72B模型的表現，證明記憶管理有時比堆參數更划算。

對正在打造長上下文agent pipeline的工程師而言，這篇論文的教訓很具體：把記憶操作變成可觀察、可追蹤的一級動作，並用去重、upsert取代append-only的紀錄方式，能大幅減少重複與繞圈，這對需要長期累積上下文、又要求可稽核的系統設計特別值得參考。

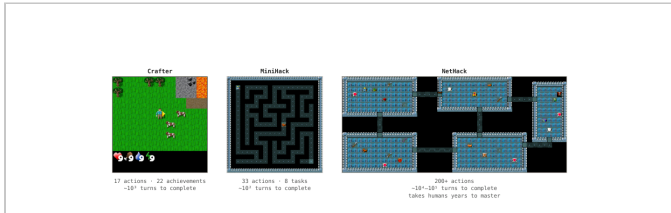


Figure 2: Long-horizon game environments for evaluating memory skills. All three environments are stochastic worlds, making each episode unique and minimizing the influence of pretraining knowledge. Crafter is an open-world survival game with crafting, combat, and resource management. MiniHack pre

延伸閱讀【1】

可回溯監督

把Agent的執行過程，變成一棵可以分叉重來的Git樹

當一個agent需要被更高階的meta-agent監督、修正甚至回滾時，現有環境通常只給transcript和環境快照，逼得開發者自己拼湊工具。Shepherd[2]的野心是把整個執行過程定義成Task、Effect、Scope、Execution Trace四個一級原語，讓agent的一舉一動都能像Git commit一樣被檢視、分叉與回滾。

靠著copy-on-write設計，fork一次的延遲固定在134-147毫秒，和映像檔大小無關，比docker commit快5倍。搭配meta-agent監督後，CooperBench上兩個agent合作的成功率從28.8%衝到54.7%，幾乎逼

近solo模式的上限，證明監督層能實質補齊多agent協作的能力落差。

這套『執行即可回溯』的模型，和醫療器材軟體要求的可追溯性與變更控制幾乎是同一套語言。若能讓agent對關鍵資料的每次操作都以可分叉、可重播的task形式執行，並讓輸出先落在待核准的retained output，等於把『AI建議、人類核准』的法規要求直接寫進系統架構，而非事後拼湊的審查層。

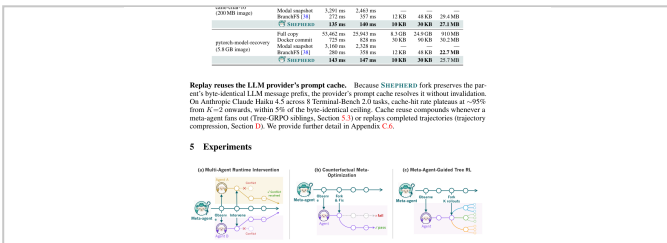


Figure 2: Headline results from the three meta-agent applications built on SHEPHERD: (A) runtime supervision on CooperBench, (B) counterfactual meta-optimization (CRO), and (C) meta-agent-guided Tree-GRPO on Terminal-Bench 2.0.

延伸閱讀【2】

模型瘦身潮

當推論不再需要打電話回雲端：模糊判斷被編譯進23MB小模型

本週有兩篇論文從不同方向逼近同一個目標：讓強大的AI能力離開昂貴的雲端API，塞進裝置或本地環境。Program-as-Weights[3]把log過濾、格式修復這類『模糊函數』直接編譯成23MB的LoRA權重，而Gemma 4[4]則用encoder-free架構與量化版本，把多模態理解塞進2.3B到31B的裝置端模型陣容裡。

PAW的0.6B interpreter在FuzzyBench上準確率73.78%，反而超越直接prompt 32B模型的68.70%，推論記憶體只要1/50；Gemma 4的26B MoE模型每次推論只活化3.8B參數，表現卻媲美397B的Qwen 3.5，搭配MTP drafter推測解碼還能再加速最多3倍。兩者殊途同歸：小，不再等於打折。

對同時身兼AI工程師與AI醫材工程師的讀者，這個趨勢的現實意義是：病歷、檢驗報告這類敏感資料的模糊判斷任務，未來有機會不必離開院內網路就能跑，既降低雲端API依賴與成本，也少一層資料外流的合規風險——值得列入下一輪技術選型的評估清單。

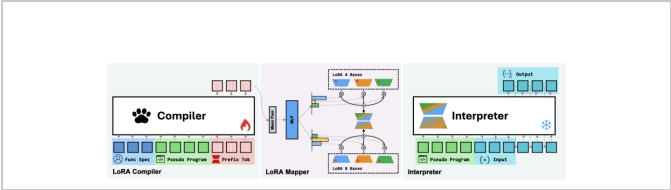


Figure 2: Text-to-LoRA instantiation of PAW (Section 3.2). Left. The trained LoRA compiler reads the function specification, the pseudo-program produced by an off-the-shelf prompted pseudo compiler Cp (not depicted), and a fixed sequence of learned prefix tokens; it emits prefix-position hidden stat

延伸閱讀 [3] [4]

02 重點學術文獻

文獻 [1]

記憶不是架構，是可以訓練的技能——AutoMem

Shengguang Wu, Hao Zhu, Yuhui Zhang et al. · 引用數 0 · <https://arxiv.org/abs/2607.01224>

- 問題 LLM agent面對上千步的長時程任務時，記憶管理向來被當成固定架構模組手工設計，鮮少被視為可學習、可優化的技能。
- 方法 研究團隊把讀寫、搜尋、建檔等記憶操作提升為與遊戲動作平起平坐的一級行為，再用meta-LLM審閱完整執行紀錄，迭代改寫記憶schema，並篩選優質記憶決策訓練出獨立的LoRA記憶專家。
- 結果 在MiniHack上，僅優化記憶就讓Qwen2.5-32B的過關率從7.5%衝到27.5%，翻了3.67倍。
- 限制 NetHack上絕對進度仍只有1.85%，顯示極端複雜任務的提升仍然有限。

一個32B模型在NetHack裡反覆走進同一條死巷，牠不是不會打怪，而是不記得三十步前已經試過這條路——這正是Stanford團隊想解決的問題：記憶，不該只是一塊固定大小的暫存區。

他們先把記憶操作（讀、寫、搜尋、建檔）變成agent可以自主決定、且能被追蹤的一級動作，而不是寫死在程式裡的固定流程。接著讓Claude Opus扮演meta-LLM，逐一審閱完整的episode紀錄，診斷agent到底在哪個環節反覆繞圈、重複寫入或搜尋落空，再據此改寫記憶檔案的schema與prompt。

光靠改寫schema還不夠，他們發現真正厲害的記憶決策藏在agent自己表現最好的那些episode裡，於是篩選這些片段，只微調一個獨立的LoRA記憶專家，遊戲決策的權重保持凍結。這個設計讓兩個模型共享對話歷史卻各司其職，也讓32B的模型不必真的長成72B就能打贏更大的對手。

三個環境的進步幅度分別是1.89倍（Crafter）、3.67倍（MiniHack）、3.74倍（NetHack），疊加記憶專家訓練後，效能已逼近Claude Opus 4.5與Gemini 3.1 Pro Thinking這類前沿模型。更值得注意的是副作用指標：無效繞圈動作減少32-65%，重複寫入減少68-83%，NetHack的每步輸入token甚至因為去重設計降了95%。這代表對長時程agent而言，把記憶管好，可能比把模型做大更划算。

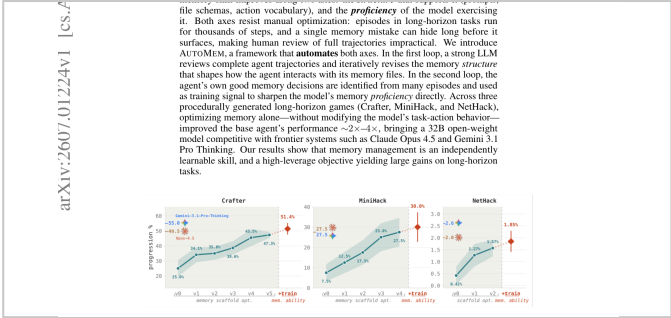


Figure 1: Memory skill optimization with Qwen2.5-32B-Instruct. Starting from a base agent equipped with file-system memory (v0), AUTOMEM progressively improves performance through memory scaffold optimization (v0–v5/v4/v2), followed by memory proficiency training (+train) that yields further gains o

缺陷與限制

1. 依賴強力審閱模型 整套優化流程仰賴Claude Opus等昂貴模型當meta-LLM審閱者，成本與可及性論文並未討論。
2. 僅驗證遊戲環境 三款測試都是探索加資源管理類程序化遊戲，能否遷移到寫程式、辦公自動化等真實長任務仍未知。
3. 極端任務進展仍低 NetHack絕對過關率僅1.85%，遠低於人類需數年才能精通的門檻，顯示方法對最複雜任務效果有限。

觀點：從工程角度看，這只是多加了一個LoRA模組；但如果記憶真的能被訓練成技能，或許意味著未來的agent不再需要更大的context window，而是需要更會『整理筆記』的自己。

文獻 [2]

把Agent的一舉一動變成可以回溯的Git commit——Shepherd

Simon Yu, Derek Chong, Ananjan Nandi et al. · 引用數 0 · <https://arxiv.org/abs/2605.10913>

- 問題 當agent需要被另一個更高階的meta-agent監督、修正或回滾時，現有環境只給transcript和環境快照，逼得開發者自己拼湊ad hoc工具。
- 方法 Shepherd把整個執行過程定義成四個一級原語（Task/Effect/Scope/Execution Trace），用copy-on-write讓agent與環境可以像Git一樣被分叉、合併、回滾，並以Lean做過形式化驗證。
- 結果 Fork/revert延遲固定在134-147毫秒，在5.8GB映像檔上比完整複製快192倍；搭配meta-agent監督後CooperBench的pair pass rate從28.8%衝到54.7%。
- 限制 三個應用案例作者自陳『並非窮盡』，且model call等不可逆效果仍只能記錄稽核、無法真正回滾。

想像一個團隊裡有兩個agent同時在改同一份程式碼，其中一個開始跑偏——過去你只能看著log乾著急，Shepherd想讓你直接把牠的執行過程當成一棵可以分叉、可以砍掉重練的commit樹。

他們的出發點是functional programming：如果agent的每一步都能被當成有型別輸入輸出的函式，那整個執行過程就能像資料一樣被檢視、替換、組合。於是Task定義agent的行為契約，Effect把每個model call、tool call拆成意圖與結果兩階段的事件，未落地前都能被攔截或丟棄。

光有事件紀錄還不夠回滾，他們遇到的難題是如何讓fork/revert快到能在生產環境用。答案是Scope用copy-on-write原子性地複製agent加環境的狀態，再用類Git的Execution Trace把每個過去版本都變成可checkout的分支，共用底層儲存，才把fork延遲從幾十秒壓到一百多毫秒。

最直觀的數字是速度：在5.8GB的映像檔上，fork一次只要143毫秒，比完整檔案複製快192倍，比docker commit快5倍——這代表『同一個起點嘗試多種策略』的成本大幅降低。更關鍵的是監督效果：CooperBench上加入Opus 4.7 meta-agent監督後，兩個agent合作的成功率從28.8%拉到54.7%，幾乎逼近solo模式的57.2%上限。這說明監督層不只是保險，還能實質補齊agent協作的能力落差。

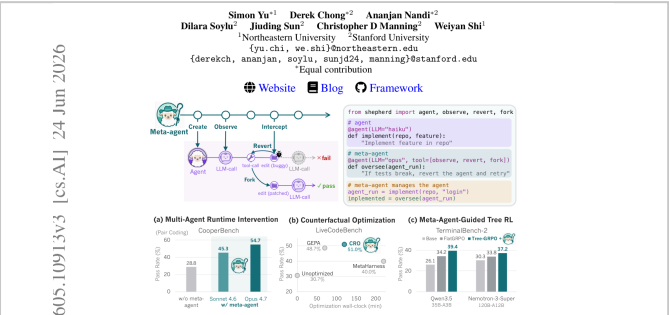


Figure 1: SHEPHERD meta-agents. Top: A meta-agent optimizes an agent's execution trace. Bottom: Results from three meta-agents: (A) runtime intervention; (B) meta-optimization; (C) Tree-GRPO.

缺陷與限制

- 1. 應用案例有限 runtime supervision、CRO、Tree-RL三個案例作者自陳只是可行性證明，未涵蓋更廣泛的meta-agent場景。
- 2. 不可逆效果無法真回滾 寄信、外部API呼叫等副作用只能記錄供稽核，系統邊界仍需開發者自行處理。
- 3. 監督成本未細算 meta-agent本身的token與監督開銷只在附錄簡單提及，缺乏完整的成本效益模型。

觀點：從技術上看這是一套執行環境；但對需要可追溯、可重現的產業而言，它可能是把『AI建議』與『人類核准』的法規要求直接寫進系統架構的第一塊磚。

文獻【3】

把模糊判斷『編譯』成23MB的離線小模型——Program-as-Weights

Wentao Zhang, Liliana Hotsko, Woojeong Kim et al. · 引用數 0 · <https://arxiv.org/abs/2607.02512>

問題 log過濾、修復格式錯誤JSON這類難以寫死規則的『模糊函數』，開發者常常每次都呼叫遠端大模型處理，犧牲了本地性、可重現性與成本。

方法 Program-as-Weights把自然語言規格編譯成LoRA權重：一個未訓練的pseudo compiler先把規格改寫成含範例的偽程式，再由訓練過的LoRA compiler動態生成LoRA注入凍結的小型interpreter。

結果 0.6B的interpreter在FuzzyBench上達73.78%準確率，超越直接prompt 32B模型的68.70%，推論記憶體只要約1/50。

限制 在長篇結構化生成任務如1m2LaTeX上，LoRA版本明顯輸給prefix-tuning前身（0.181 vs 0.391）。

如果每次判斷一則log是不是告警，都要打一次API call給大模型，帳單和延遲都會失控——這篇論文的野心是把這類判斷『編譯』成一個23MB、可以離線跑的小模型，像編譯程式一樣，編譯一次、到處執行。

他們先遇到的問題是：規格常常寫得又髒又模糊，直接拿去訓練效果很差。於是先用一個沒訓練過的pseudo compiler，把原始規格改寫成帶有輸入輸出範例的『偽程式』，相當於先把使用者的模糊意圖翻譯成機器看得懂的中介語言，再往下一步走。

第二步才是真正的重頭戲：訓練一個LoRA compiler，讀進規格與偽程式，透過共享基底加小型MLP動態生成一組LoRA參數，即時注入一個凍結的小型interpreter（如Qwen3 0.6B）。他們發現換掉compiler（改用視覺版4B模型）就能讓同一個0.6B interpreter直接處理圖表理解任務，顯示compiler和interpreter是可以分開換的兩塊積木。

最關鍵的數字是效率：0.6B interpreter的準確率（73.78%）反而超過直接prompt 32B模型（68.70%），推論記憶體只要約1.2GB，是後者的1/50，在MacBook M3上冷啟動僅0.48秒。面對含錯字與語意模糊的規格，準確率只掉3.7個百分點，顯示pseudo-program階段確實有『去噪』效果。但在長篇結構化任務上，它明顯不如prefix-tuning，說明這套壓縮並非萬能，長上下文範例仍會擠壓小模型有限的容量。

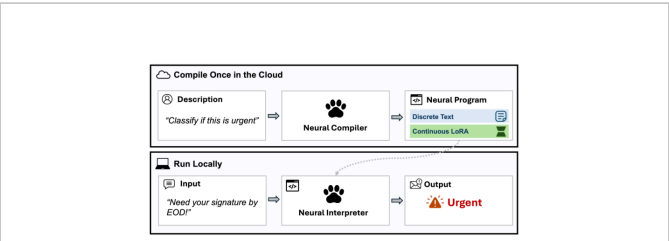


Figure 1: Overview of the Program-as-Weights paradigm. Top: compile once in the cloud. A natural-language description of a fuzzy function (here, "classify if this is urgent") is fed to a neural compiler, which produces a neural program. Bottom: run locally. A small frozen neural interpreter loads the

名詞解釋

LoRA（低秩適應） LoRA凍結模型原始大權重矩陣，只在旁側插入兩個低秩小矩陣訓練，訓練完可把乘積加回原權重，幾乎不增加推論延遲。相較於在KV cache前插入可學習虛擬token的prefix-tuning，LoRA直接調整權重本身的行為，是目前最主流的參數高效微調技術之一。

缺陷與限制

- 1. 長文本生成較弱 1m2LaTeX等長篇結構化任務上，LoRA版本準確率明顯低於prefix-tuning前身，顯示範例過長會擠壓小型interpreter的容量。
- 2. 訓練資料自產自足 FuzzyBench完全由gpt-5.2生成並自我驗證，效能上限被生成模型自身的96.09%準確率所侷限。
- 3. 表達力更強的變體反而更差 多個理論上更精細的LoRA mapper設計實測效果更差，作者坦承缺乏清楚的理論解釋。

觀點：從技術上看這只是換了一種微調方式；但如果模糊判斷真能像程式一樣被編譯、被版本控制，或許意味著雲端LLM API不再是處理『不確定性』的唯一入口。

文獻 〔4〕

31B模型登頂開源Arena，但真正的野心是塞進裝置裡——Gemma 4

Gemma Team (Google DeepMind) · 引用數 4 · <https://arxiv.org/abs/2607.02770>

問題 要在裝置端部署的小尺寸限制下，同時具備多模態理解與強推理能力，還要解決長context帶來的KV cache記憶體爆炸，幾乎是互相拉扯的三個目標。

方法 Gemma 4提供dense與MoE兩種架構，為12B模型設計encoder-free統一架構直接吃原始音訊與圖像patch，並用5:1 local-to-global attention比例與KV cache共享壓縮長文本記憶體。

結果 31B dense模型在Arena人類評測Elo 1451，登頂開源模型；E4B在RULER 128k準確率86.6%，大幅超越前代Gemma 3 27B的66.0%。

限制 論文未提供encoder-free架構相對於獨立encoder版本的直接消融對照，難以量化『拿掉encoder』本身的效能損失。

當多數頭條還在比誰的旗艦模型分數更高時，Gemma 4真正想贏的戰場其實是你的手機和邊緣裝置——一個能塞進裝置、還聽得懂你說話的模型。

他們面對的難題是：獨立的vision/audio encoder雖然效果好，卻讓小尺寸模型的部署成本居高不下。於是12B模型乾脆拿掉獨立encoder，直接把40毫秒的原始音訊片段與圖像patch投影進LLM的embedding空間，讓同一套權重同時處理文字、影像與聲音。

另一個難題是長context下KV cache記憶體爆炸，他們的解法是把5:1甚至4:1的local-to-global attention比例，搭配p-RoPE位置編碼與KV cache共享，並讓global layer的keys等於values，組合起來把global KV cache砍掉37.5%。同時釋出QAT量化版本與MTP drafter head，讓推測解碼在不犧牲品質下加速。

03 學術動向

當AI開始被股票市場定價，以及穿戴式健康模型的新答案

本週唯一一篇聚焦金融的文獻，反而提供了觀察AI擴散最誠實的尺規。[5]用OpenRouter超過380兆token的真實付費紀錄建構『AI Factor』，發現曝險越高的公司，股票報酬溢酬越明顯——value-weighted多空策略每週報酬達64.1個基點，經風險因子調整後仍有56個基點。溢酬集中在closed-source模型、資深用戶與長prompt這些『重度使用』的邊際，顯示市場獎勵的不是誰在用AI，而是誰用得深。

更值得工程師注意的一個數字是：agentic token佔比從2024年趨近於零，一路衝上樣本末期超過50%——這與本週其他幾篇論文不約而同指向的方向（agent正在取代單次問答成為主流用法）彼此呼應，只是[5]是從市場定價的角度給出獨立佐證。技能層級的分析也呼應了這個趨勢：互動與溝通類技能的曝險顯著為正，分析與操作控制類技能反而是負相關。

另一則來自Google Research的研究動態同樣值得一提：SensorFM用超過一兆分鐘、五百萬人的無標籤穿戴式感測資料自監督訓練出通用生理表徵模型，遷移到心血管、代謝、睡眠等35項健康預測任務，且模型規模與資料規模同步擴大帶來近乎線性的效能提升——把它接進個人健康代理人後，臨床醫師評分顯示效果與真實實測數據幾乎無統計差異，對長期關注醫療AI的讀者是條值得追蹤的路線。

31B dense模型在Arena人類評測拿下Elo 1451，是開源dense模型第一名；26B-A4B的MoE版本雖然只活化3.8B參數，表現卻能媲美397B的Qwen 3.5。長context上E4B在128k RULER測試達86.6%準確率，比前代Gemma 3 27B的66.0%高出一大截，GraphWalks F1分數也從32.8跳到50.9。這些數字合在一起說明：小模型不再只是『能用但打折扣』的選項，在特定任務上已經逼近甚至超越更大的模型。



論文首頁預覽：[arxiv_2607.02770](https://arxiv.org/abs/2607.02770)

缺陷與限制

- 缺乏encoder消融實驗** encoder-free 12B模型沒有和帶encoder版本的直接對照，難以單獨判斷拿掉encoder犧牲了多少效能。
- 安全評測只有定性描述** 論文5.3節安全評測僅稱『顯著優於前代』，沒有具體數字或標準化分數可供對照。
- 非英語覆蓋不均** 訓練資料截至2025年1月，部分語言如中文的字元錯誤率仍明顯偏高（0.136-0.235）。

觀點：從規格表上看這是一份技術報告；但對需要離線、低延遲、保護病患資料的醫療AI場景而言，它可能是把大模型能力真正搬進院內網路的一塊拼圖。

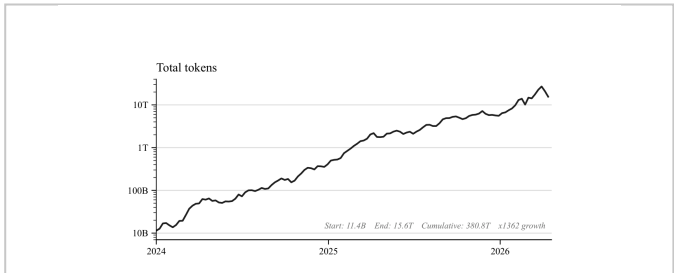


Figure 1: Weekly Total Tokens The figure plots weekly total-token consumption on a log scale. Total tokens are prompt tokens plus completion tokens. The in-figure annotation reports the first-week level, final observed-week level, cumulative total-token consumption of 380.8 trillion, and growth mul

新詞櫥窗

GRPO / Tree-GRPO GRPO是LLM對齊訓練中常用的強化學習法，對同一提示採樣一組rollout，用組內相對優劣當優勢訊號，免去額外訓練價值網路。Tree-GRPO讓meta-agent在rollout中動態選擇分叉時間點，讓共享前綴的兩條軌跡各自走出不同結果，把原本只在任務結束才有的稀疏獎勵，轉換成逐步的信用分配訊號。

Mixture-of-Experts (MoE) MoE把單一前饋層換成多個平行『專家』子網路，由路由器依輸入動態挑選一小部分專家實際運算，其餘不參與。這讓模型擁有很大的總參數量以提升知識容量，但每次推論的活化參數量遠小於總量，運算成本接近小模型，是近期大模型維持效率的主流架構選擇。

推測解碼 (Speculative Decoding) / MTP Drafter 推測解碼用一個較小較快的草稿模型先猜測未來幾個token，再由大模型在單次前向傳播中平行驗證，猜對的直接採用、猜錯才重新生成，品質與大模型逐字生成完全一致。MTP drafter則是不另訓練獨立小模型，而是從大模型自身隱藏狀態預測後續token的實作方式。

量化感知訓練 (QAT) QAT在訓練或微調階段就插入偽量化運算，讓模型權重與活化值提前『適應』低精度量化會產生的數值誤差，使最終量化成int4、int8等低位元部署時，準確度損失遠小於單純訓練後量化。這讓大模型能以更小的記憶體佔用與更低延遲跑在裝置端或資源受限環境。

參考文獻

[1] Shengguang Wu, Hao Zhu, Yuhui Zhang et al. , 「AutoMem: Automated Learning of Memory as a Cognitive Skill」, <https://arxiv.org/abs/2607.01224> (引用 0)

[2] Simon Yu, Derek Chong, Ananjan Nandi et al. , 「Shepherd: Enabling Programmable Meta-Agents via Reversible Agentic Execution Traces」, <https://arxiv.org/abs/2605.10913> (引用 0)

[3] Wentao Zhang, Liliana Hotsko, Woojeong Kim et al. , 「Program-as-Weights: A Programming Paradigm for Fuzzy Functions」, <https://arxiv.org/abs/2607.02512> (引用 0)

[4] Gemma Team, Sherif El Abd, Vaibhav Aggarwal et al. , 「Gemma 4 Technical Report」, <https://arxiv.org/abs/2607.02770> (引用 4)

[5] Nicola Borri, Yukun Liu, Aleh Tsyvinski , 「AI Premium」, <https://arxiv.org/abs/2606.30583>

週知快報 | 週報自動生成系統 | 引用來源見上列 | 語音導讀約 15 分鐘